

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_04_RecursiveAlgorithms

Exercise 1: YADAV Anshuman Krishna

Recursive Comment Thread Traversal

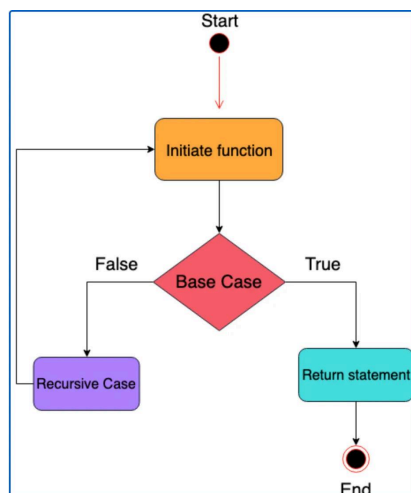
1. Base Structure

```
STRUCTURE CommentNode
comment_id : String
user_id : String
content : String
timestamp : Integer
likes : Integer
replies : List CommentNode
END STRUCTURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

2. Recursive Traversal



II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

A. display_thread

```
PROCEDURE display_thread(comment, level)
indentation <- CreateString(" ", level)
PRINT indentation + comment.content

FOR EACH reply IN comment.replies DO
display_thread (reply, level + 1)
END FOR
END PROCEDURE
```

(had to google this one)

B. count_total_comments

```
FUNCTION count_total_comments(comment) -> Integer
total <- 1
FOR EACH reply IN comment.replies DO
total <- total + count_total_comments(reply)
END FOR
RETURN total
```

Time Complexity : $O(1)$

Space Complexity : $O(d)$, d is max depth

C. total_likes

```
FUNCTION total_likes(comment) -> Integer
sum_likes <- comment.likes

FOR EACH reply IN comment.replies DO
sum_likes <- sum_likes + total_likes(reply)
END FOR
RETURN sum_likes
```

Time Complexity : $O(n)$, since we visit every node

Space Complexity : $O(d)$, recursive call stack

D. find_deepest_reply

```
FUNCTION find_deepest_reply(comment) -> Integer
max_depth <- 0

FOR EACH reply IN comment.replies DO
depth <- find_deepest_reply(reply)
```

```
IF depth > max_depth THEN
max_depth <- depth
END IF END FOR
RETURN max_depth + 1
```

Time Complexity : $O(n)$, since we visit every node

Space Complexity : $O(d)$, recursive call stack

3. Implement Search Functionality

A. search_by_user

```
FUNCTION search_by_user(user_id, comment) -> List
results <- empty list
```

```
IF comment.user_id = user_id THEN
results.Add(comment)
END IF
```

```
FOR EACH reply IN comment.replies DO
child_results <- search_by_user(user_id, reply)
FOR EACH res IN child_results DO
results.Add(res)
END FOR END FOR
RETURN results
```

(had to google this one too)

B. contains_keyword

```
FUNCTION contains_keyword(keyword, comment) -> Boolean
IF comment.content.Contains(keyword) THEN
RETURN TRUE
END IF
```

```
FOR EACH reply IN comment.replies DO
IF contains_keyword(keyword, reply) = TRUE THEN
RETURN TRUE
END IF END FOR
RETURN FALSE
```

Time Complexity : $O(n)$, worst case,

Space Complexity : $O(d)$, recursive call stack

4. Implement Deletion with Cascade

```
FUNCTION delete_comment(comment_id, thread) -> CommentNode
IF thread.comment_id = comment_id THEN
RETURN NULL
END IF
```

```
new_replies <- empty list
```

```
FOR EACH reply IN thread.replies DO
filtered_reply <- delete_comment(comment_id, reply)
IF filtered_reply != NULL THEN
new_replies.Add(filtered_reply)
END IF
END FOR
```

```
thread.replies <- new_replies
RETURN thread
```

Complexity Analysis Questions:

1. What is the time complexity of traversing an entire comment thread with n -total comments?

Answer:

The time complexity will be $O(n)$ because we are dealing with a tree-like structure, hence we will be visiting each single node once to perform various operations (like counting, displaying, etc.)

2. How does recursion depth relate to the comment nesting structure? What happens with very deep threads (1000+ levels)?

Answer:

The recursion depth is equal to the thread's nesting level. Every time someone replies to a pre-existing reply, the function calls itself one level deeper. For a thread with 1000+ levels, it will open 1000+ nested functions at the same time, which can exceed a machine's built-in memory limits and can cause a stack-overflow crash.

3. Compare memory usage between recursive and iterative stack-based implementations?

Answer:

Both the implementations would have a space complexity of $O(d)$, mathematically, where d will be the depth of the thread.

The recursive approach uses the execution call stack of the system which stores function parameters, return addresses and local variables. Hence, it is rigid and also has strict size limits.

what we already knew,

If we're building a platform like Reddit or X where threads can go many levels deep, iterative is a better choice to avoid crashes.

The iterative approach uses a manual stack data structure which is stored on the system's heap. While it still takes memory to store the nodes, heap memory is vastly larger than call stack memory, making the iterative version much safer for extremely deep comment threads without crashing.

Code Related Queries

Advanced Complexity Analysis

(n : total number of comments in the thread)

(d : maximum depth/nesting level of the thread)

- **Traversal & Metrics (display_thread, count_total_comments, total_likes):** Time $O(N)$, Space $O(D)$.
Because we perform a DFS traversal, we must visit every single node to display it or sum its values, meaning time scales linearly with the total number of comments.
- **Max Depth (find_deepest_reply):** Time $O(N)$, Space $O(D)$.
Even though we only want the depth, we cannot know which branch is deepest without exploring all of them, therefore, we still touch every node.
- **Searching (search_by_user, contains_keyword):** Time $O(N)$, Space $O(D)$.
`contains_keyword` might exit early in a best-case scenario ($O(1)$ if the root has the keyword), but the worst-case technically forces us to check the very last node. `search_by_user` uses extra $O(K)$ memory to store the results list, where k is the number of matching comments found.

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

- **Tree Pruning (delete_comment):** Time $O(N)$, Space $O(D)$.
We rebuild (recursively) the replies arrays. We touch every node to check its ID against the target. The space complexity is $O(D)$ for the stack, plus a little extra memory at each level to generate the temporary new_replies lists before reassigning them.

Test Cases

Input	Expected Output	Reason
Normal Count & Likes: Tree with 5 nodes. Likes: 50, 10, 5, 20, 100.	Total Comments: 5 Total Likes: 185	Tests if the recursive functions properly visit every single branch and accumulate the integers correctly all the way back up to the root.
Max Depth Calculation: Root -> Reply1 -> Reply1_1 -> Reply1_1_1	Max Depth: 4	Ensures the algorithm successfully compares sibling branches and accurately returns the longest path length to the bottom.
Keyword Search: Search for "cool" which is located at depth level 4.	Returns: True	Verifies that the search algorithm traverses the entire nested structure deeply enough to find a match and properly bubbles the True flag back up.

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Cascading Deletion: Delete Comment "2" (which has two child replies).	Comment 2, 4, and 5 are removed.	Tests the cascade logic. By returning None for Comment 2, it inherently orphans its children, successfully pruning that entire branch from the data structure.
Edge Case (Root Deletion): Delete Comment "1" (the top-level root).	Returns: None	Verifies that if the original thread starter is deleted, the entire tree collapses gracefully without throwing an error.

Reflection

Working through these recursive algorithms made it clear for us as to why recursion is the good standard for tree-like data structures. A comment thread with nested replies is naturally recursive. Writing this using standard while loops and manual stacks would have required drastically more code and complexity just to keep track of where we were in the tree.

However, we also saw a few cons of recursion. Because every nested reply adds a new frame to the system's execution stack, a heavily debated post with thousands of nested replies could easily trigger a Stack Overflow crash.

Exercise 2: MAHALINGAM Nithees

Recursive Content Aggregation with Divide & Conquer

1. Post Structure

Structure Post
post_id

```
user_id
content_preview
time_stamp
likes
comments
shares
engagement_score
End Structure
```

2. Divide & Conquer

Part A : Maximum Engagement

```
Function max_engagement(posts, left, right)
  If left = right
    return posts[left]
  mid <- (left + right) / 2
  left_max <- max_engagement(posts, left, mid)
  right_max <- max_engagement(posts, mid + 1, right)
  If left_max.engagement_score > right_max.engagement_score
    return left_max
  Else
    return right_max
End
```

Part B : Total and Average

Total

```
Function sum_engagement(posts, left, right)
  If left = right
    return posts[left].engagement_score
  mid <- (left + right) / 2
  left_sum <- sum_engagement(posts, left, mid)
  right_sum <- sum_engagement(posts, mid + 1, right)
  return left_sum + right_sum
```

End

Sum

```
Function average_engagement(posts, left, right)
  total <- sum_engagement(posts, left, right)
  n <- right - left + 1
  return total / n
End
```


Part C : Count by Threshold

Function count_above_threshold(posts, left, right, threshold)

```
If left = right
  If posts[left].engagement_score > threshold
    return 1
  Else
    return 0
mid <- (left + right) / 2
left_count <- count_above_threshold(posts, left, mid, threshold)
right_count <- count_above_threshold(posts, mid + 1, right, threshold)
return left_count + right_count
End
```

Part D : Merge Sort by Engagement

Merge Function

```
Function merge(posts, left, mid, right)
  Create arrays L and R
  Copy posts[left..mid] into L
  Copy posts[mid+1..right] into R
  i <- 0, j <- 0, k <- left
  While i < length(L) AND j < length(R)
    If L[i].engagement_score ≤ R[j].engagement_score
      posts[k] <- L[i]
      i <- i + 1
    Else
      posts[k] <- R[j]
      j <- j + 1
    k <- k + 1
  Copy remaining elements of L
  Copy remaining elements of R
End
```

Merge Sort

```
Function merge_sort(posts, left, right)
  If left < right
    mid <- (left + right) / 2
    merge_sort(posts, left, mid)
    merge_sort(posts, mid + 1, right)
    merge(posts, left, mid, right)
End
```

3. Peak Hours Analysis

```
Function find_peak_hour(likes, left, right)
  If left = right
    return left
  mid <- (left + right) / 2
  If likes[mid] < likes[mid + 1]
    return find_peak_hour(likes, mid + 1, right)
  Else
    return find_peak_hour(likes, left, mid)
End
```

Complexity Analysis Questions:

1. What is the time complexity of recursive max_engagement? Prove it.

Answer:

Recurrence: $T(n) = 2T(n/2) + O(1)$

Using Master Theorem: $T(n) = O(n)$

The array is divided into two halves and each element is compared once.

2. Compare recursive merge sort ($O(n \log n)$) with iterative insertion sort ($O(n^2)$) for sorting 10,000 posts.

Answer:

Merge Sort

$O(n \log n)$

$10000 \times \log_2(10000)$

$10000 \times 14 = 140,000$ operations

Insertion Sort

$O(n^2)$

$10000^2 = 100,000,000$ operations

3. What is the recursion depth for merge sort on n elements?

Answer:

$O(\log n)$

Array is divided into halves repeatedly until size = 1.

4. For find_peak_hour, why can we use binary search-style recursion? What property must the array satisfy?

Answer:

The array must satisfy:

- It should have at least one peak
- Typically follows a bitonic pattern (increasing then decreasing)

Binary search works because:

- If $\text{arr}[\text{mid}] < \text{arr}[\text{mid}+1]$, peak lies on right
- Else peak lies on left

Code Related Queries

Complexity Analysis

max_engagement:

$$T(n) = 2T(n/2) + O(1) \\ = O(n)$$

sum_engagement:

$$T(n) = 2T(n/2) + O(1) \\ = O(n)$$

count_above_threshold:

$$T(n) = 2T(n/2) + O(1) \\ = O(n)$$

merge_sort:

$$T(n) = 2T(n/2) + O(n) \\ O(n \log n)$$

find_peak_hour:

$$T(n) = T(n/2) + O(1) \\ = O(\log n)$$

Function	Time Complexity	Space Complexity
max_engagement	$O(n)$	$O(\log n)$
sum_engagement	$O(n)$	$O(\log n)$
count_above_threshold	$O(n)$	$O(\log n)$
merge_sort	$O(n \log n)$	$O(n)$

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

peak_hour	$O(\log n)$	$O(\log n)$
-----------	-------------	-------------

Reflection:

This implementation uses divide-and-conquer to efficiently analyze social media engagement data. Functions like max, sum, and count break the problem into smaller parts, achieving $O(n)$ time, while merge sort ensures efficient sorting in $O(n \log n)$. The peak hour function uses a binary search approach with $O(\log n)$ complexity. Overall, this method improves performance and scalability, though edge cases must be handled carefully.

Test Cases:

Input	Expected Output	Description
4 posts: (50,25,10), (100,80,20), (40,20,5), (90,70,10)	Max = Post2 (320), Sum = 845, Avg = 211.25, Above 200 = 2, Sorted = [95,150,280,320]	Normal case
1 post: (10,5,2)	Max = same post, Sum = 24, Avg = 24	Single post
0 posts	No posts available	Empty input
3 posts: (10,5,0), (10,5,0), (10,5,0)	Max = any, Sorted = same values	Equal engagement
Threshold = 1000	Above Threshold = 0	High threshold
Threshold = 0	Above Threshold = n	Low threshold
Posts in reverse order	Sorted correctly ascending	Merge sort correctness
Likes: [5,8,12,25,30,28,15]	Peak index = 4 (30)	Peak in middle
Likes: [30,20,10]	Peak index = 0	Peak at start
Likes: [5,10,20,40]	Peak index = last index	Peak at end
Likes: [10]	Peak index = 0	Single element
Likes: []	No data	Empty likes

Exercise 3: SARA VANAN Arun Prasath

Converting Recursion to Iteration with Explicit Stack

1. Recursive Flatten

```
FUNCTION flatten_recursive(comment):  
  result = [comment]  
  FOR each reply IN comment.replies:  
    result = result + flatten_recursive(reply)  
  RETURN result
```

2. Iterative Flatten (Using Stack)

```
FUNCTION flatten_iterative(comment):  
  stack = [(comment, "START")]  
  result = []  
  while stack is not empty:  
    node, state = POP(stack)  
    if state == "START":  
      result.append(node)  
      stack.push((node, "DONE"))  
      for reply in reversed(node.replies):  
        stack.push((reply, "START"))  
  return result
```

3. Tail Recursive Counting

```
FUNCTION count_comments_tail(comment, count):  
  count = count + 1  
  for reply in comment.replies:  
    count = count_comments_tail(reply, count)  
  return count
```

4. Iterative Counting (Loop Version)

```
FUNCTION count_comments_loop(comment):  
  stack = [comment]  
  count = 0
```

```
while stack is not empty:  
    node = POP(stack)  
    count = count + 1  
    for reply in node.replies:  
        stack.append(reply)  
return count
```

Complexity Analysis Questions:

1. For a thread with depth d and total n comments, compare maximum stack size needed for:

- Recursive implementation
- Iterative with explicit stack

Answer:

Recursive implementation: uses system call stack $\rightarrow$ max size = $O(d)$

Iterative (explicit stack): uses manual stack $\rightarrow$ max size = $O(d)$

2. When would you prefer recursive version? When iterative?

Answer:

Use Recursive:

- When the structure is naturally hierarchical (like nested comments/tree)
- Code is simpler and easier to read

Use Iterative:

- When depth can be very large (to avoid stack overflow)
- When you need better control over memory

3. How does tail recursion conversion reduce stack usage? Can all recursive functions be made tail recursive?

Answer:

- Tail recursion means the last step is the recursive call
- It can be optimized to use less stack space ($O(1)$)
- But not all recursive functions can be changed into tail recursion
- Because sometimes work is needed after the recursive call

4. Explain the state machine approach—why is it necessary for functions that process nested structures?

Answer:

- It keeps track of what step we are in
- Useful for handling nested structures step by step
- In recursion, this is automatic
- In iteration, we must manage it manually

5. In a production social network with user-generated nested comments, which approach would you choose and why?

Answer:

- Prefer iterative approach
- Handles large and deep data safely
- Avoids stack overflow issues
- More reliable for real-world applications

Code Related Queries

Reflection:

This exercise shows how recursion can be replaced using an explicit stack. The recursive approach is simple and matches the structure of nested comments, but it depends on the system stack.

The iterative version gives better control over execution by manually managing the stack. It is more reliable when dealing with deep nesting, which is common in real-world applications like social media comment threads.

This comparison helps in understanding how both approaches work internally and when to use each one based on constraints like memory and scalability.

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Complexity Analysis:

- **Flatten (Recursive & Iterative):**

Time: $O(n)$, each comment is visited once

Space: $O(d)$, due to recursion stack or explicit stack

- **Counting (Recursive & Iterative)**

Time: $O(n)$, all comments are processed

$O(d)$, depends on depth of the structure

Test Cases:

Input Structure	Output	Reason
Single comment	Count = 1	Base case
One level replies	Count = n	Simple traversal
Deep nested chain	Count = n	Tests max depth
Empty replies	Only root	No children to process
Large nested tree	Correct flatten list	Tests scalability

Final Integration Question: Engagement Analytics with Nested Comments

To build this feature, we need to combine all the linear and non-linear data structures we explored this semester. Here's our architectural breakdown of how we'd design this system.

1. Working

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

Our goal is to build a dashboard that ranks the most viral posts and identifies our most active users. The pipeline works in four distinct stages:

- **Recursive Algorithms:** First, we dive into the data. Since comments can have endless replies, we use recursion to explore the nested comment trees of every post. During this deep dive, we calculate a total engagement score (counting every nested reply) and grab the user IDs of everyone who commented.
- **Set Operations:** As our recursive function grabs user IDs, it dumps them into Hash Sets. By doing an Intersection across the sets of different posts, we can instantly find the "super users" who are actively commenting across multiple threads.
- **Priority Queues:** We take the total engagement scores and push every post into a Priority Queue (a Max Heap). This auto-ranks the content, putting the posts with the highest engagement at the top.
- **Doubly Linked Lists:** Lastly, we pull the top posts from the Priority Queue and link them into a Doubly Linked List. This powers the UI, letting the analyst swipe through the trending posts with instant $O(1)$ navigation.

2. Which operations

- **Recursive:** Exploring the nested comments has to be recursive. Since a reply can have another reply, the tree structure demands recursive calls that dive deep into the branches.
- **Iterative:** Everything else in this pipeline is iterative. Inserting posts, running the Set intersections, and navigating the Doubly Linked List are all sequential ops handled by regular `while` or `for` loops.

3. Handling potential stack overflow

If a post sparks a comment thread that's thousands of levels deep, our recursive traversal will open thousands of nested calls and crash the server.

To fix this, we'd build a safety valve. We could cap the recursion depth at something safe, like level 20 or 25. Anything deeper would require the user to click "Load More Replies", which clears the stack and starts fresh. Or we could rewrite

II.2415 Advanced Algorithms and Programming

Lab 4: Linear Data Structures: Team T24

the traversal iteratively using a manual Stack, storing the data on the heap instead of the fragile execution stack.

4. Performance considerations

If this scales to millions of posts, our current architecture would choke if we tried to process it all in real-time.

The Priority Queue bottleneck: Pushing millions of posts dynamically takes $O(N \log N)$ time, way too slow for a live page load. We'd move this to the background, calculating the top 1000 posts overnight and caching them.

The Set Operations bottleneck: Storing millions of user IDs in memory for intersections would consume tons of RAM. We'd need to shard the DB or switch to a probabilistic data structure (like a Bloom Filter) to estimate overlaps without storing every ID.

The DLL advantage: The Doubly Linked List will perform fine at scale since navigation is always $O(1)$. But we'd only load small batches (maybe 50 at a time) to avoid crashing the client's browser.

thank you.